

Richieste HTTP in JS/TS

la fetch API

All'interno dell'ambiente browser per effettuare richieste HTTP verso un server si utilizza una **API** denominata **fetch**.

Essa consta in sostanza di un'unica funzione, appunto la funzione **fetch**

fetch(url: string, init?: RequestInit)

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Request line

- **HTTP method**
specifica quale **azione** deve essere eseguita sul server. Ci sono varie azioni (dette **method** o **verb**) HTTP possibili, per il momento ci considereremo i due «di base»:
 - **GET** – il server deve fornire informazioni al client
 - **POST** – il server deve elaborare in qualche modo i dati che sta ricevendo dal client
- **destinazione**
URL della richiesta (**completa** o con solo il **path assoluto** – si assume che lo hostname sia quello che sta ricevendo la richiesta)
- **versione**
la versione utilizzata del protocollo HTTP

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81

{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Header

Molte coppie (name, value) sono impostate **di default** dal browser quando la richiesta viene formulata.

Più comunemente impostati dal programmatore:

- **Accept**
Tipi di dati accettati in **risposta**
- **Content-Type**
Tipo di dati contenuti nel **body**
- **User-Agent**
Informazioni sul tipo di **client** che sta formulando la richiesta

La HTTP Request

Un po' più in dettaglio

HTTP Request

```
POST /api/contacts HTTP/1.1
Host: localhost:8001
User-Agent: curl/7.64.1
Accept: */*
Content-Type: application/json
Content-Length: 81
```

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "johndoe@example.com"
}
```

Body

Le richieste di tipo **GET** non hanno un **body** perché il loro obiettivo è **ottenere** dei dati, non inviarne.

Le richieste di tipo **POST** hanno come obiettivo **inviare** dati e dunque hanno un **body** tipicamente di uno dei seguenti tipi:

application/x-www-form-urlencoded

dati contenuti in un form compilato dall'utente e inviato dal browser

application/json

dati elaborati da una app client-side e inviati dall'app stessa

Richieste HTTP in JS/TS

la fetch API

All'interno dell'ambiente browser per effettuare richieste HTTP verso un server si utilizza una **API** denominata **fetch**.

Essa consta in sostanza di un'unica funzione, appunto la funzione **fetch**

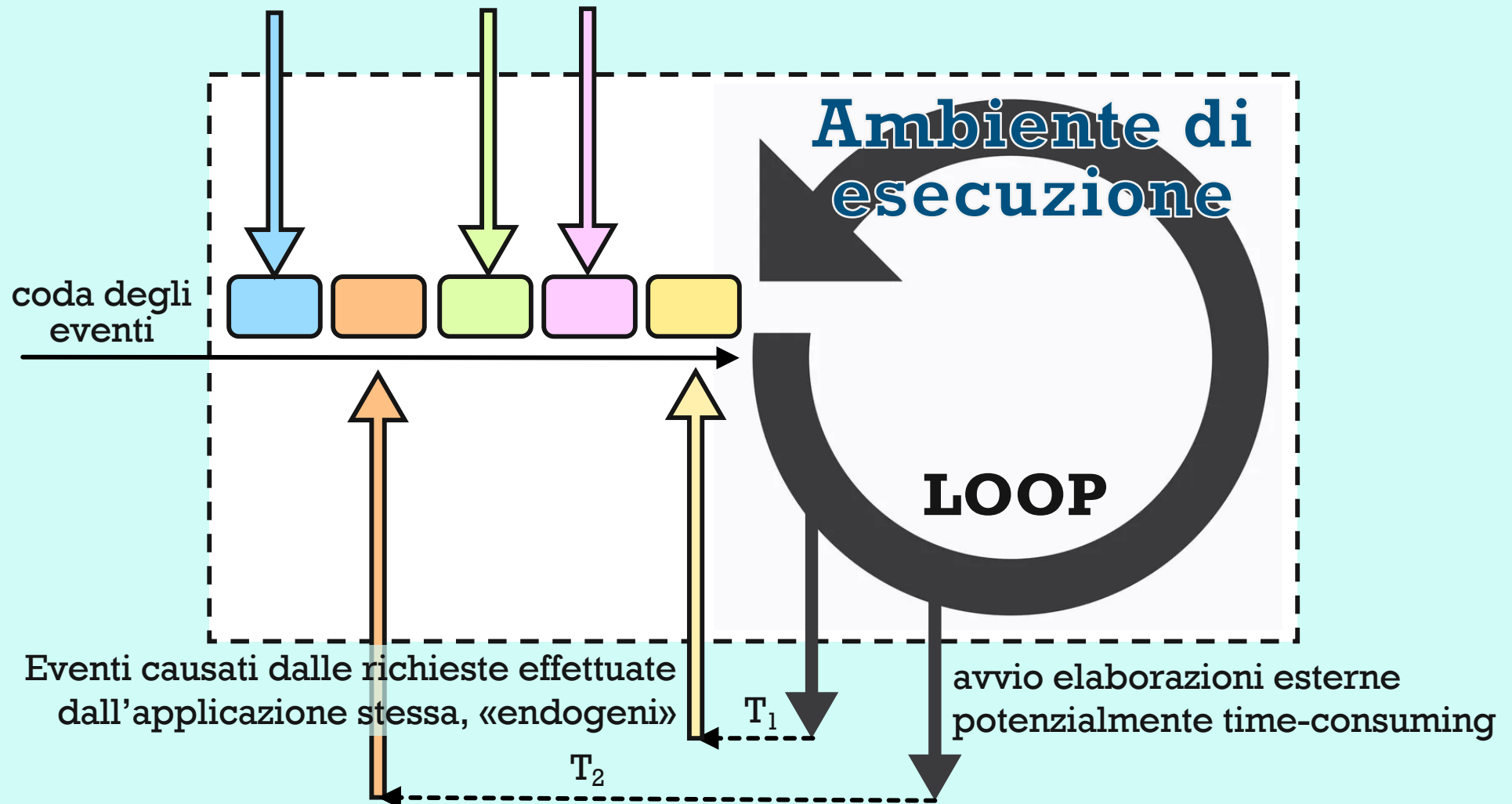
fetch(url: string, init?: RequestInit)

```
interface RequestInit {  
  method?: string;  
  headers?: { [key: string]: string };  
  body?: string | null;  
}
```

Javascript e gli eventi

asynchronous single-thread event loop

Eventi «spontanei» generati all'esterno, «esogeni»



Richieste HTTP in JS/TS

la fetch API

All'interno dell'ambiente browser per effettuare richieste HTTP verso un server si utilizza una **API** denominata **fetch**.

Essa consta in sostanza di un'unica funzione, appunto la funzione **fetch**

fetch(url: string, init?: RequestInit): Promise<Response>

Fetch API - I

La **fetch API** va a sostituire una precedente API (denominata XMLHttpRequest, dal nome della struttura dati che principalmente la caratterizzava) per formulare richieste HTTP verso un server. La «novità» (ormai non più tanto nuova, risale al 2015) è che la **fetch API** ingloba il paradigma AJAX.

AJAX

Precedentemente all'introduzione del paradigma AJAX, se anche il codice Javascript effettuava una richiesta HTTP, la risposta a tale richiesta veniva **intercettata dal browser** che usava i dati contenuti nella risposta per visualizzare una nuova pagina (con conseguente reset dell'ambiente di esecuzione). Il paradigma AJAX prevedeva invece che, a fronte di una richiesta HTTP effettuata da codice Javascript, la risposta venisse intercettata dall'ambiente di esecuzione e ricevuta dal programma stesso tramite un'opportuna callback. In questo modo il programma in esecuzione poteva utilizzare i dati contenuti nella risposta per modificare il DOM e dunque l'aspetto della pagina, senza forzare il ricaricamento, che sarebbe stato problematico per l'esecuzione del programma stesso. Prima dell'esecuzione della fetch API, era necessario installare opportune librerie per poter usare AJAX. La fetch API invece lo include in automatico: una richiesta HTTP effettuata tramite la funzione **fetch** torna sempre al programma che l'ha generata.

AXIOS

Axios è una delle librerie che nell'epoca pre-fetch permettevano di utilizzare il paradigma AJAX. A differenza di altre, però, è ancora utilizzata in progetti nuovi per due ragioni:

- La formulazione della richiesta nel codice è **lievemente** più semplice che con **fetch**
- La sua API permette di formulare richieste HTTP anche in ambienti di esecuzione diversi dall'ambiente-browser, mentre la **fetch API** è disponibile solo nell'ambiente-browser

Se volete potete usare AXIOS per il vostro progetto, ma non è obbligatorio conoscerla.

Fetch API - II

I parametri della funzione **fetch** permettono di costruire la richiesta HTTP come meglio si crede. Vediamo i punti salienti, quelli cioè indispensabili o quasi per un'applicazione client-side.

- Il parametro **url** specifica la risorsa che si vuole ottenere. Può essere una URL esterna (dunque specificare anche il dominio), assoluta o relativa. Nel caso di una URL assoluta o relativa, il dominio si intende essere il medesimo di provenienza della richiesta. Nel caso di una URL assoluta il punto di partenza per cercare la risorsa sarà la cartella radice del dominio; nel caso di una URL relativa il punto di partenza per cercare la risorsa sarà la location della pagina html in cui viene eseguito lo script che effettua la richiesta.
- Il parametro opzionale **init** deve essere un oggetto che rispetti l'interface **RequestInit**. Questa interface elenca moltissime proprietà per lo più opzionali, ci limitiamo a prendere in esame quelle più frequentemente utilizzate:
 - **method**: il metodo HTTP (di default è "GET" ma potremo, ad esempio, scegliere di indicare "POST").
 - **headers**: un oggetto usato come dizionario associativo di coppie chiave/valore; le chiavi sono i possibili header HTTP (quindi, "Content-Type", "User-Agent", "Accept", ecc...) a cui si può scegliere di assegnare un valore
 - **body**: per i method HTTP che **inviano** dati al server (POST su tutti, e probabilmente è l'unico che vedremo, ma ce ne sono altri), uno stream che rappresenta i dati in questione in un formato corrispondente al Content-Type dichiarato nell'header. Per i nostri scopi sarà sempre una **string**.

Spesso, se si sta effettuando una richiesta GET, il parametro **init** può interamente essere omesso, in quanto i valori di default di tutte le opzioni risultano adeguati.

Fetch API - III

La funzione **fetch** è **asincrona**: avvia un'elaborazione (ossia, la trasmissione della richiesta in rete) e restituisce immediatamente il controllo al programma, che dovrà in qualche modo essere avvertito quando la risposta è arrivata (o quando scade un timeout che segnala un'attesa eccessiva, possibile spia di una connessione non funzionante o di un server che non risponde).

L'arrivo della **risposta HTTP** (o del timeout) è quello che abbiamo chiamato un evento **endogeno**, ossia provocato dal programma in esecuzione. Nel momento in cui parte la richiesta, **siamo certi** che questo evento accadrà.

Avevamo visto come uno dei modi «classici» di gestire questa situazione in JS/TS sia l'uso di una **funzione di callback**: la funzione che esegue l'operazione asincrona (ossia, la richiesta HTTP) richiede, fra i parametri, una funzione atta a gestire l'arrivo della risposta o del timeout, che verrà chiamata nel momento in cui l'evento si verifica.

Come mai allora la funzione fetch non prende nessuna funzione come argomento?

Nella stessa edizione di Javascript in cui è stata introdotta la **fetch API** è stata introdotta anche la **Promise API**: essa implementa un meccanismo più sofisticato, e meno suscettibile di errori di programmazione, per gestire gli eventi endogeni, ossia quegli eventi che si verificano a seguito di una richiesta partita dal codice stesso. Promise significa «promessa» ed è ciò che una operazione asincrona restituisce, non un dato, bensì la **promessa** di un dato che arriverà nel futuro.

La **Promise API** non ha, di per sé, a che vedere con le richieste HTTP; è un'API generica che può essere usata in qualsiasi operazione asincrona. La **fetch API** tuttavia la sfrutta.

Promise

Alcuni problemi del meccanismo di callback

```
function update() {  
  
    const dbCb = (dati: number[]) => {  
        aggiornaVisualizzazione(dati);  
    }  
  
    const webCb = (info: string) => {  
        stampaInformazioni(info);  
        leggiDBGrosso("bla bla", dbCb);  
    }  
  
    caricaDaWebLento("un.indirizzo.boh", webCb);  
  
}
```

Promise

Alcuni problemi del meccanismo di callback

```
function update(cb: (dati: number[]) => void) {  
  
    const dbCb = (dati: number[]) => {  
        aggiornaVisualizzazione(dati);  
        cb(dati);  
    }  
  
    const webCb = (info: string) => {  
        stampaInformazioni(info);  
        leggiDBGrosso("bla bla", dbCb);  
    }  
  
    caricaDaWebLento("un.indirizzo.boh", webCb);  
  
}
```

Promise

Alcuni problemi del meccanismo di callback

```
function update(cb: (dati: number[], info: string) => void) {  
  
    const dbCb = (dati: number[], info: string) => {  
        aggiornaVisualizzazione(dati);  
        cb(dati, info);  
    }  
  
    const webCb = (info: string) => {  
        stampaInformazioni(info);  
        leggiDBGrosso("bla bla", (dati: number[]) => dbCb(dati, info));  
    }  
  
    caricaDaWebLento("un.indirizzo.boh", webCb);  
  
}
```

I problemi del meccanismo di callback

Per capire le difficoltà che il meccanismo standard delle callback può causare ai programmatori, e la facilità con cui si possono commettere errori nell'usarlo, analizziamo l'esempio sulla slide precedente.

Il **corpo** della funzione **update** effettua le seguenti operazioni:

- Richiede una risorsa web, all'indirizzo **un.indirizzo.boh**
- Quando le informazioni richieste arrivano, le stampa a video, e successivamente legge dei dati piuttosto voluminosi da un database
- Quando arrivano anche i dati dal database, aggiorna la visualizzazione.

In gioco ci sono due operazioni asincrone, ciascuna con la sua callback: la richiesta della risorsa via web, e la lettura da database. Siccome la seconda deve avvenire dopo la prima, deve essere effettuata all'interno della prima callback.

La prima cosa che possiamo notare sin da ora è che il codice è scritto **a rovescio** rispetto alle tempistiche con cui avvengono le cose: questo è dovuto al fatto che dobbiamo scrivere le callback **prima** di invocarle, ma l'ordine di esecuzione è stabilito dall'**invocazione** delle funzioni, non dalla loro **definizione**!

Immaginiamo ora che la funzione **update** debba venir invocata all'interno di un programma. Potrebbe essere necessario (ma non sappiamo se lo è) che il resto del programma sappia quando **update** è terminata, per compiere qualche altra operazione. L'unico modo che abbiamo per avere questa possibilità è far sì che anche **update** richieda una callback, da chiamare nel momento opportuno...

Si vede rapidamente quanto è complicato far sì che questa nuova callback riceva a cascata i risultati delle elaborazioni precedenti. Ad ogni ulteriore livello di indirezione il meccanismo diventa più complicato, ed è sempre più difficile tener traccia della reale sequenza in cui nel codice avvengono le diverse operazioni, persino quando (come in questo caso) sono completamente sequenziali.

Il meccanismo delle callback e l'elevato tasso di asincronicità porta ad una sorta di dispersione del codice per cui la catena di cause ed effetti diventa difficile da ricostruire; una volta in esecuzione, non si sa più chi era ad aver chiamato la callback e le conseguenze di un errore diventano pressoché impossibili da tracciare all'indietro alla ricerca della causa.

Promise<T>

Cos'è una Promise?

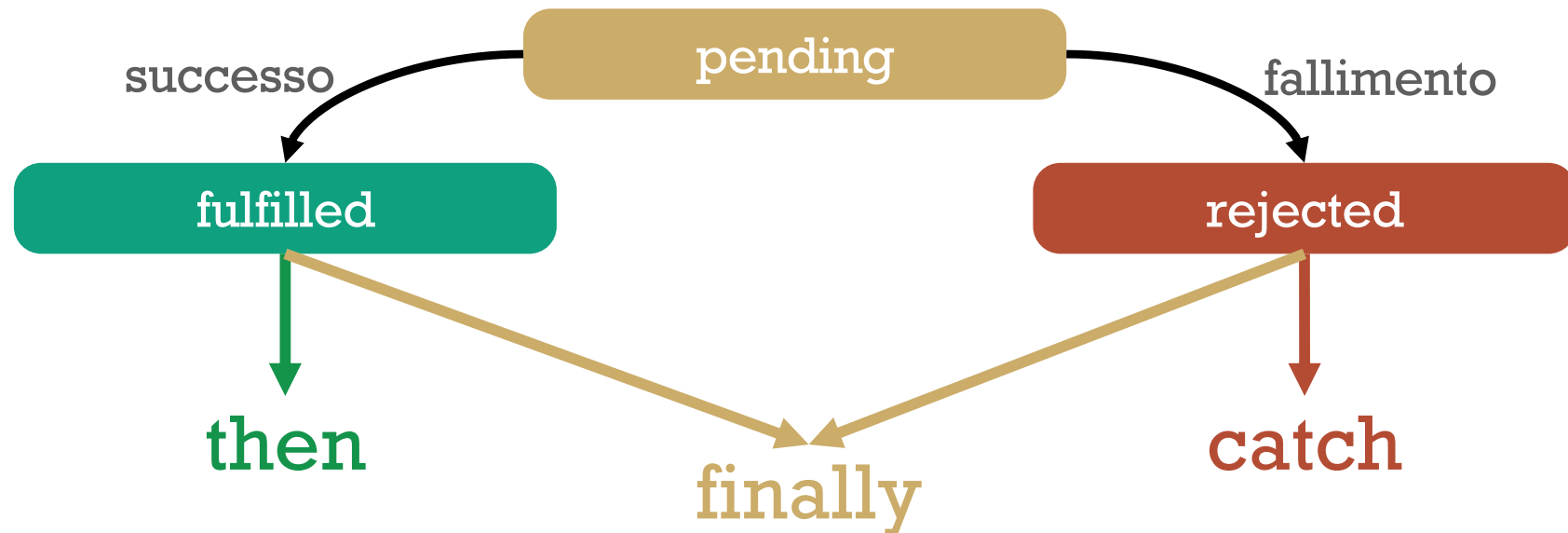
Un oggetto **Promise<T>** rappresenta la **promessa** di un dato T.



Promise<T>

Cos'è una Promise?

Un oggetto **Promise<T>** rappresenta la **promessa** di un dato T.



Promise – Cosa sono

Cos'è una **Promise**? Una **Promise** è un oggetto che rappresenta l'esito (che potrebbe essere ancora da venire, o già completato) di una operazione asincrona. Abbiamo detto che una funzione asincrona restituisce immediatamente il controllo al chiamante. Con il meccanismo delle Promise, lo fa restituendo un oggetto ben preciso, appunto di tipo Promise.

Il tipo Promise è un *generic*: la funzione asincrona restituirà un oggetto **Promise<T>** se al completamento dei lavori ci si aspetta di ottenere un valore di tipo **T**.

Il chiamante non **delegherà** la funzione asincrona ad eseguire il proprio codice di gestione del risultato (come avveniva con le callback), ma **aggancerà** tale codice all'oggetto Promise ottenuto.

La sostanza di ciò che accade è la stessa: le **Promise** non servono ad ottenere comportamenti particolari prima impossibili, servono a rendere più leggibile il codice e a far sì che l'ordine della **scrittura** delle istruzioni sia il più possibile affine al relativo ordine di **esecuzione**.

Un oggetto Promise può essere in uno di tre stati:

- **Pending**: L'operazione è ancora in corso, la Promise dunque non è risolta e il risultato effettivo non è disponibile. Tutti gli oggetti Promise nascono in stato **pending**.
- **Fulfilled**: L'operazione è terminata con successo, la Promise è mantenuta e il risultato effettivo è disponibile.
- **Rejected**: L'operazione è fallita, la Promise non può essere mantenuta, il risultato effettivo non è disponibile e mai lo sarà, potrebbe però essere disponibile un oggetto contenente informazioni sull'errore che si è verificato.

Promise – Come si usano

Il codice che **invoca** una funzione asincrona riceve in risposta un oggetto Promise. Vediamo come può usarlo per eseguire una funzione (quella che prima era la callback) in seguito al completamento dell'operazione.

In generale dato un oggetto di tipo Promise è possibile **agganciare** ad esso una funzione che viene eseguita nel momento in cui lo stato passa da **pending** a **fulfilled** o **rejected**. Il **modo** in cui la funzione viene agganciata determina se viene eseguita in caso di successo (fulfilled), fallimento (rejected) o entrambi

- Chiamando il metodo **then** su un oggetto Promise agganciamo una funzione che verrà chiamata in caso di successo, con il dato ottenuto
- Chiamando il metodo **catch** su un oggetto Promise agganciamo una funzione che verrà chiamata in caso di fallimento, con l'oggetto di errore
- Chiamando il metodo **finally** su un oggetto Promise agganciamo una funzione che verrà chiamata in ogni caso

Tutti e tre i metodi restituiscono una nuova Promise il cui stato di successo o fallimento dipende da una combinazione di:

- lo stato di successo o fallimento della promise originale
- quello che è accaduto durante l'esecuzione della funzione agganciata

L'oggetto «promesso» dalla nuova Promise è quello restituito dalla funzione agganciata. A questa nuova Promise potremo agganciare un'altra funzione, che gestisca questo nuovo dato. È quindi possibile creare delle catene di funzioni che vengono chiamate successivamente e infine restituire la Promise ottenuta. Il chiamante potrà attaccare ad essa le proprie funzioni, e restituirla a sua volta.

Promise – Come si usano

Il codice che **invoca** una funzione asincrona riceve in risposta un oggetto Promise. Vediamo come può usarlo per eseguire una funzione (quella che prima era la callback) in seguito al completamento dell'operazione.

In generale dato un oggetto di tipo Promise è possibile **agganciare** ad esso una funzione che viene eseguita nel momento in cui lo stato passa da **pending** a **fulfilled** o **rejected**. Il **modo** in cui la funzione viene agganciata determina se viene eseguita in caso di successo (fulfilled), fallimento (rejected) o entrambi

- Chiamando il metodo **then** su un oggetto Promise agganciamo una funzione che verrà chiamata in caso di successo, con il dato ottenuto
- Chiamando il metodo **catch** su un oggetto Promise agganciamo una funzione che verrà chiamata in caso di fallimento, con l'oggetto di errore
- Chiamando il metodo **finally** su un oggetto Promise agganciamo una funzione che verrà chiamata in ogni caso

Tutti e tre i metodi restituiscono una nuova Promise il cui stato di successo o fallimento dipende da una combinazione di:

- lo stato di successo o fallimento della promise originale
- quello che è accaduto durante l'esecuzione della funzione agganciata

L'oggetto «promesso» dalla nuova Promise è quello restituito dalla funzione agganciata. A questa nuova Promise potremo agganciare un'altra funzione, che gestisca questo nuovo dato. È quindi possibile creare delle catene di funzioni che vengono chiamate successivamente e infine restituire la Promise ottenuta. Il chiamante potrà attaccare ad essa le proprie funzioni, e restituirla a sua volta.

Promise Then

```
const p: Promise<T> = funzioneAsincrona(...);
```

```
const p2: Promise<U> = p.then((dati: T) => {  
    // istruzioni di elaborazione dei dati  
    ...  
    const someresult: Promise<U> = altrafunAsincrona(dati);  
    return someresult;  
});
```

- la funzione che associamo tramite **then** a una **Promise<T>** può ricevere un singolo parametro di tipo **T** (oppure può non prendere parametri)
 - essa verrà chiamata solo se e quando la **Promise** passerà da **pending** a **fulfilled**
- il risultato di invocare **x.then(fun)** su **x:Promise<T>** è un nuovo oggetto **y:Promise<U>**
 - **U** coincide con il tipo restituito da **fun** (eventualmente void)
 - solo nel caso particolare in cui **fun** restituisce un oggetto **z:Promise<X>**, l'oggetto risultante **y** è a sua volta una **Promise<X>** (perché una promessa di una promessa è una promessa...)
- se **x** ha successo, anche **y** diventa **fulfilled**
 - ma: se la funzione **fun** restituisce una terza Promise **z**, **y** avrà successo o fallirà in conseguenza del successo o fallimento di **z**, con i medesimi risultati o messaggi di errore
- se **x** fallisce dando un errore **err**, anche **y** diventerà **rejected** con lo stesso errore **err**

Promise Catch

```
const prom: Promise<number[]> = funzioneAsincrona(...);  
const prom2 = prom  
    .then((dati: number[]) => {...})  
    .catch((err: Error) => { //gestione errore;  
        ...  
    });
```

- la funzione che associamo tramite **catch** a una **Promise<T>** può ricevere un singolo parametro che rappresenta l'errore generato (oppure può non prendere parametri)
 - essa verrà chiamata solo se e quando la **Promise** passerà da **pending** a **rejected**
- il risultato di invocare **x.catch(fun)** su **x:Promise<T>** è un nuovo oggetto **u:Promise<U>**
 - **U** coincide con il tipo restituito da **fun** (eventualmente void)
 - solo nel caso particolare in cui **fun** restituisce un oggetto **z:Promise<X>**, l'oggetto risultante **u** è a sua volta una **Promise<X>** (perché una promessa di una promessa è una promessa...)
- se l'errore generato in **x** viene dunque correttamente gestito, **u** può diventare **fulfilled**
 - ma: se la funzione **fun** restituisce una terza Promise **z**, **u** avrà successo o fallirà in conseguenza del successo o fallimento di **z**, con i medesimi risultati o messaggi di errore
- se **fun** lancia (**throw**) un errore **err2**, **u** diventerà **rejected** con errore **err2**
- di solito catch si concatena dopo then.

Promise Finally

```
const prom: Promise<number[]> = funzioneAsincrona(...);
const prom2 = prom
    .then((dati: number[]) => {...}).catch((err: Error) => {...})
    .finally(() => {
        ...
    })
```

- la funzione che associamo tramite **finally** a una **Promise<T>** verrà quando la **Promise** uscirà dallo stato **pending**, sia che essa risulti **fulfilled**, sia che essa risulti **rejected**
- il risultato di invocare **x.finally(fun)** su **x:Promise<T>** è un nuovo oggetto **w:Promise<T>**
 - se **fun** lancia un errore o restituisce una Promise che fallisce, **w** fallisce a sua volta
 - altrimenti **w** si comporterà in modo equivalente a **x** e il risultato verrà ignorato.
- Anche **finally** permette di proseguire con il concatenamento

Promise

Combinare più operazioni asincrone

Congiunzione

Date N operazioni asincrone $op1...opN$, la loro congiunzione è un'operazione composta C tale che

- C si considera conclusa con successo solo se e quando tutte le $op1...opN$ sono terminate con successo

Disgiunzione

Date N operazioni asincrone $op1...opN$, la loro disgiunzione è un'operazione composta C tale che

- C si considera conclusa con successo solo se e quando una fra $op1...opN$ termina con successo

Concatenazione

Date N operazioni asincrone $op1...opN$, la loro concatenazione è un'operazione composta C tale che

- $op(i)$ ha inizio solo se e quando $op(i-1)$ si è conclusa con successo
- C si considera conclusa con successo solo se e quando opN è terminata con successo

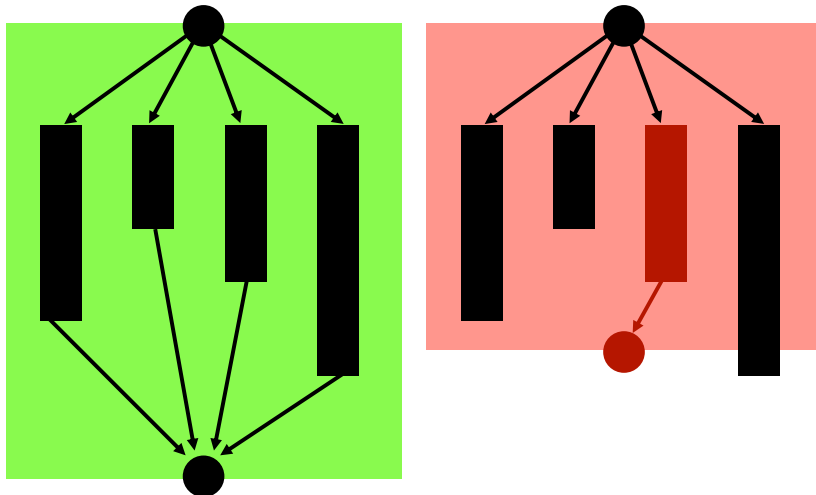
Promise

Combinare più operazioni asincrone

Congiunzione

Date N operazioni asincrone $op1...opN$, la loro congiunzione è un'operazione composta C tale che

- C si considera conclusa con successo solo se e quando tutte le $op1...opN$ sono terminate con successo



```
const p1: Promise<T1> = op1(...);
const p2: Promise<T2> = op2(...);
const p3: Promise<T3> = op3(...);
const conj: Promise<[T1, T2, T3]>
  = Promise.all([p1, p2, p3])
```

- **conj fulfilled**
 - se e quando **p1, p2, p3** sono tutte fulfilled
 - risultato: l'array dei risultati di **p1, p2, p3**
- **conj rejected**
 - non appena una fra **p1, p2, p3** è rejected
 - risultato: l'errore della promise che ha fallito per prima

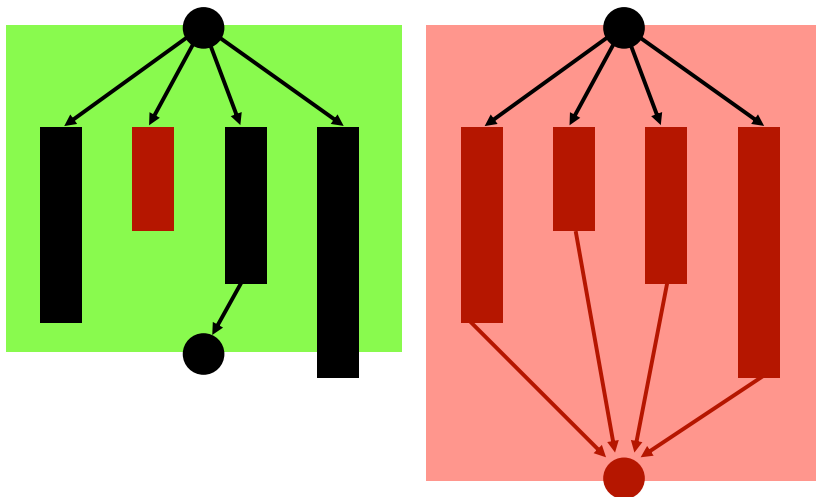
Promise

Combinare più operazioni asincrone

Disgiunzione

Date N operazioni asincrone $op1 \dots opN$, la loro disgiunzione è un'operazione composta C tale che

- C si considera conclusa con successo solo se e quando una fra $op1 \dots opN$ termina con successo



```
const p1: Promise<T1> = op1(...);
const p2: Promise<T2> = op2(...);
const p3: Promise<T3> = op3(...);

const disj: Promise<T1|T2|T3>
  = Promise.any([p1, p2, p3])
```

- **disj fulfilled**
 - quando la prima fra $p1$, $p2$, $p3$ è fulfilled
 - risultato: quello della prima promise risolta
- **disj rejected**
 - quando e se tutte $p1$, $p2$, $p3$ risultano rejected
 - risultato: un oggetto **AggregateError** che contiene gli errori delle promise coinvolte

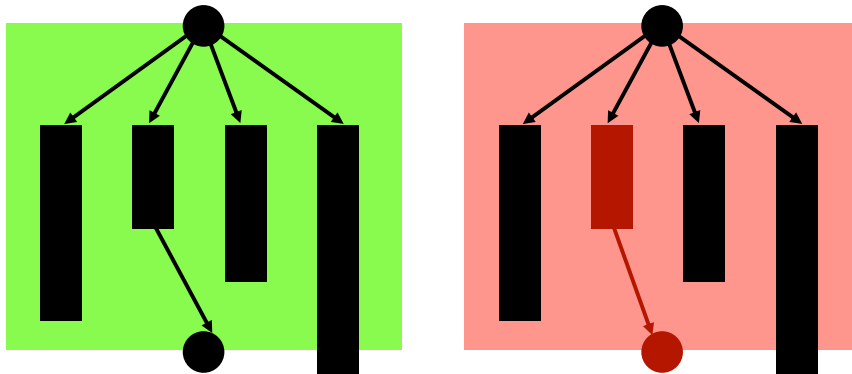
Promise

Combinare più operazioni asincrone

Competizione (variante della disgiunzione)

Date N operazioni asincrone $op1 \dots opN$, la loro disgiunzione è un'operazione composita C tale che

- C si considera conclusa con successo solo se e quando una fra $op1 \dots opN$ termina con successo prima che un'altra fallisca



```
const p1: Promise<T1> = op1(...);  
const p2: Promise<T2> = op2(...);  
const p3: Promise<T3> = op3(...);  
const comp: Promise<T1|T2|T3>  
  = Promise.race([p1, p2, p3])
```

- **disj** si resolve
 - quando la prima fra $p1, p2, p3$ si resolve
 - con il medesimo risultato della p "vincente"
- **disj fulfilled**
 - se la p "vincente" è **fulfilled**
- **disj rejected**
 - se la p "vincente" è **rejected**

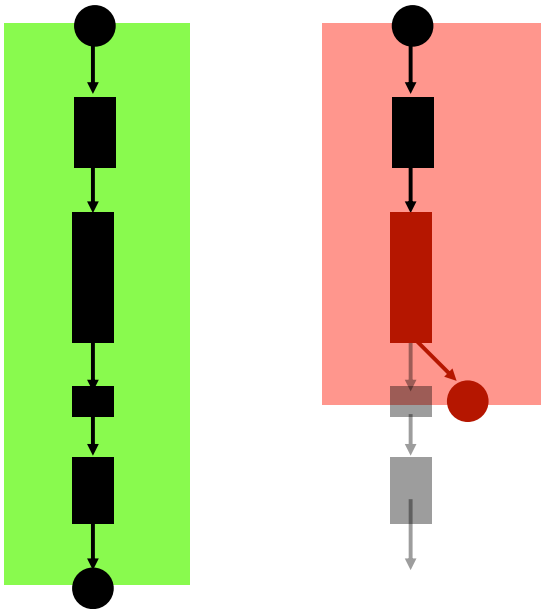
Promise

Combinare più operazioni asincrone

Concatenazione

Date N operazioni asincrone $op1 \dots opN$, la loro concatenazione è un'operazione composta C tale che

- $op(i)$ ha inizio solo se e quando $op(i-1)$ si è conclusa con successo
- C si considera conclusa con successo solo se e quando opN è terminata con successo



```
const chain: Promise<T3> = op1(...)
  .then((d: T1)=> {
    return op2(...);
  })
  .then((d: T2)=> {
    return op3(...);
  });
```

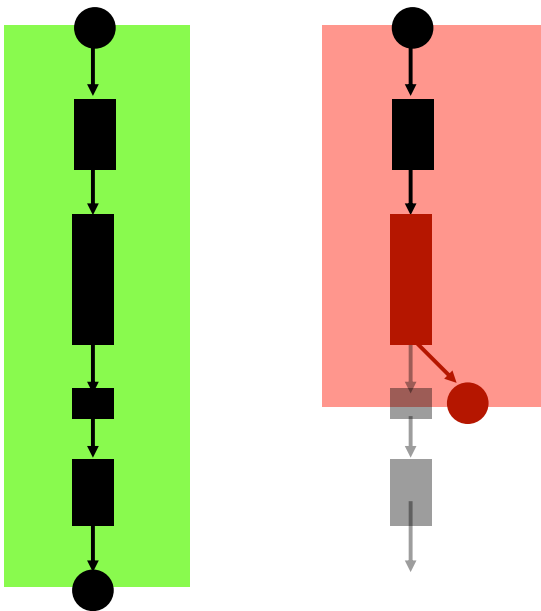
Promise

Combinare più operazioni asincrone

Concatenazione

Date N operazioni asincrone $op1 \dots opN$, la loro concatenazione è un'operazione composita C tale che

- $op(i)$ ha inizio solo se e quando $op(i-1)$ si è conclusa con successo
- C si considera conclusa con successo solo se e quando opN è terminata con successo



```
const p1: Promise<T1> = op1(...);
```

```
const p2: Promise<T2> =  
  p1.then((x: T1) => op2(...));
```

```
const p3: Promise<T3> =  
  p2.then(x: T2 => op3(...));
```

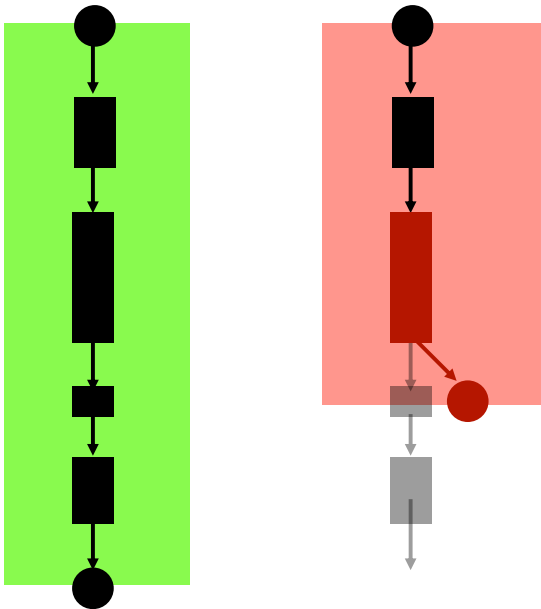
Promise

Combinare più operazioni asincrone

Concatenazione

Date N operazioni asincrone $op1 \dots opN$, la loro concatenazione è un'operazione composita C tale che

- $op(i)$ ha inizio solo se e quando $op(i-1)$ si è conclusa con successo
- C si considera conclusa con successo solo se e quando opN è terminata con successo



```
const chain: Promise<T3> =  
  op1 (...)  
    .then((d: T1) => op2 (...));  
    .then((d: T2) => op3 (...));  
  );
```

Promise

Async/Await

Le parole chiave **async** e **await** possono essere usate per semplificare la scrittura e la leggibilità di funzioni che fanno ampio uso di concatenazione fra Promise

async si applica alla **definizione** di una funzione o metodo che restituisce una `Promise<T>`

- abilita l'uso della parola chiave **await** nel codice della funzione
- al termine della funzione sarà possibile restituire un T, che verrà implicitamente e automaticamente incapsulato in una Promise

await si permette **all'invocazione** di una funzione o metodo che restituisce una `Promise<U>`

- permette di trattarla come se fosse una normale funzione che restituisce un U
- è tuttavia implicito che il codice successivo alla chiamata verrà eseguito solo alla risoluzione della `Promise<U>`, che resta implicita

async/await non aggiungono una funzionalità a TS/JS: il codice che ne fa uso può essere tradotto con una **concatenazione di then**

Promise

Async/Await

```
function someFunction(): Promise<T3> {  
  const p1: Promise<T1> = op1(...);  
  
  const p2: Promise<T2> = p1.then((d: T1)=> op2(...));  
  const p3: Promise<T3> = p2.then((d: T2)=> op3(...));  
  return p3;  
}
```

```
async function someFunction(): Promise<T3> {  
  const x1: T1 = await op1(...);  
  const x2: T2 = await op2(...);  
  const x3: T3 = await op3(...);  
  return x3;  
}
```

il formato JSON

JavaScript Object Notation

Il formato **JSON** nasce in ambiente Javascript per la **serializzazione** di oggetti principalmente ai fini della loro trasmissione in una **HTTP Connection**

Stringa JSON

valore semplice

```
3.14
```

```
true
```

```
"ciao"
```

```
null
```

oggetto

```
{  
  "chiave1": VALORE_1,  
  ...  
  "chiaveN": VALORE_N  
}
```

array

```
[  
  VALORE_1,  
  ...  
  VALORE_N  
]
```

il formato JSON

Serializzazione e Deserializzazione

Oggetto JS/TS

```
const b1 = {  
  issueId: "1-12",  
  status: "own"  
}  
const b2 = {  
  issueId: "2-7",  
  status: "own"  
}  
  
const ogg = {  
  owner: "Claudia",  
  id: 315,  
  books: [b1, b2]  
}
```

const str =
 JSON.stringify(ogg);

Stringa JSON

```
{  
  "owner": "Claudia",  
  "id": 315,  
  "books": [{  
    "issueId": "1-12",  
    "status": "own"  
  }, {  
    "issueId": "2-7",  
    "status": "own"  
  }]  
}
```

const ogg =
 JSON.parse(str);
const b1 = ogg.books[0];
const b2 = ogg.books[1];

JSON - I

Il formato JSON nasce per la **serializzazione** di oggetti Javascript. Per **serializzazione** si intende la rappresentazione di oggetti come stringhe, al fine da poterla salvare in un file di testo o inviare (come nel nostro caso) come «body» di una HTTP Request o HTTP Response.

La **serializzazione** deve essere un'operazione invertibile: l'oggetto serializzato deve poter essere **deserializzato**. Questo significa **ricostruire** l'oggetto a partire dalla stringa che lo rappresenta.

Con l'evoluzione delle applicazioni Web, l'uso di JSON si è esteso alla rappresentazione in generale di informazioni strutturate: vediamo questo tipo di utilizzo nei file **tsconfig.json** o **package.json**. Rimane il fatto che leggere e interpretare questo formato (e dunque anche questi file) a partire da codice Javascript è estremamente **semplice**.

Sebbene il formato JSON sia molto comune soprattutto nel mondo Javascript/Typescript (per esempio, il mondo Java tende a preferirgli, per gli stessi scopi, il formato XML) è evidente che, nel momento in cui lo si usa per scambiare dati in una connessione HTTP, il linguaggio utilizzato per realizzare applicazioni server-side dovrà comunque averci a che fare.

Vedremo infatti che ci sono librerie che permettono di **serializzare/deserializzare** con JSON anche oggetti Java, anche se l'operazione sarà un pochino più macchinosa. Lo vedremo per Java, perché per la server-side app useremo Java Spring Boot, ma vale lo stesso anche per altri linguaggi in cui si potrebbe sviluppare un'applicazione lato server (PHP, Python, Ruby...).

JSON può attualmente essere considerato a tutti gli effetti un formato di interscambio e rappresentazione testuale di dati **language-independent**.

JSON - II

Una **stringa JSON** può essere costituita da:

- un **valore semplice**: un **number**, una **string**, un **boolean**, oppure **null**
- un **array JSON**: un elenco di **valori JSON** separati da virgole, racchiusi fra parentesi quadre []
- un **oggetto JSON**: una sequenza di coppie **chiave: val**, separate da virgole e racchiuse fra parentesi graffe { }, dove la **chiave** è una stringa fra virgolette e **val** è un **valore JSON**

I **valori JSON** possono essere **valori semplici**, **array JSON** oppure **oggetti JSON**, quindi possiamo concludere che una **stringa JSON** è composta da un **singolo valore JSON**.

Come si nota la sintassi è molto simile a quella degli oggetti Javascript, con alcune importanti differenze:

- i valori semplici sono di tipologie più limitate (non esiste un modo per esprimere un **bigint** o un **symbol**, e non è ammesso **undefined**)
- non è possibile esprimere funzioni
- in un oggetto, le **chiavi** (che in JS sarebbero i nomi delle proprietà) vanno rappresentate come stringhe fra virgolette

JSON - III

Serializzazione

Un oggetto JS/TS può essere **serializzato** chiamando il metodo statico **JSON.stringify** che prende come argomento un oggetto, un array o un valore, e restituisce la corrispondente stringa JSON.

Il meccanismo di serializzazione esplora l'oggetto in profondità, ricorsivamente.

La serializzazione può:

- ignorare alcuni valori o proprietà di un oggetto, se non sono serializzabili: ad esempio gli **undefined** e le **funzioni** non vengono serializzati
- dare errore se la **struttura** dell'oggetto lo rende non serializzabile, in particolare se procedendo ricorsivamente ad esplorare la struttura dell'oggetto emergono riferimenti circolari.

Si pensi ad esempio ad una struttura ad albero in cui il nodo padre contiene i riferimenti ai figli, e i nodi figli contengono un riferimento al padre: emerge una circolarità che porterebbe in loop infinito l'algoritmo di serializzazione

Quindi **non tutti gli oggetti JS/TS** sono serializzabili, o per lo meno non è detto che la loro serializzazione preservi interamente le informazioni al loro interno.

La serializzazione può essere applicata ad oggetti creati a partire da una **classe**, ma l'appartenenza alla classe in questione è un'informazione che si perde; nel processo di serializzazione l'oggetto diventa a tutti gli effetti un object literal.

Deserializzazione

Una stringa JSON può essere **deserializzata** chiamando il metodo statico **JSON.parse** che prende come argomento la stringa e restituisce un object literal, un array o un valore semplice, a seconda del contenuto.

Nel caso di un oggetto, non è possibile «inserirlo» d'ufficio in una classe: è un object literal, e sappiamo che in JS/TS gli unici oggetti che appartengono a una classe sono quelli creati tramite il suo costruttore. Se però sappiamo che l'oggetto rispetta una certa **interface** possiamo tiparlo tramite essa:

```
const libro = JSON.parse(stringaDati) as Book
```

il formato JSON

JSON nel **body** di una HTTPRequest

HTTPRequest (di tipo **POST**) formulata dall'app client-side:

```
fetch(someUrl, {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify(objectToSend)  
})
```

Essendo una POST è necessario passare come secondo parametro un oggetto **RequestInit**, in modo da:

- specificare in **method** che si tratta di una richiesta POST
- specificare tramite **headers** che il **Content-Type** è **application/json**
- impostare il **body** in modo che coincida con la **serializzazione JSON** dei dati che vogliamo inviare al server.

il formato JSON

JSON nel **body** di una **HTTPResponse**

HTTPResponse proveniente dall'app server-side:

```
fetch(...)  
  .then((res: Response) => res.json())  
  .then((dati: MyExpectedDataType) => {  
    ...fa qualcosa con i dati...  
  });
```

Quando riceviamo una risposta **res** non possiamo leggere direttamente **res.body**; dobbiamo invece chiamare il metodo **res.json()** che restituisce una **Promise** dell'oggetto o valore contenuto nel **body**

- Il **body** di una **Response** non è una **string**!!! Bensì un **ReadableStream**
- Questo perché all'arrivo della **HTTPResponse** i dati potrebbero non essere ancora interamente pervenuti (si pensi al caso in cui la risorsa richiesta è un video o un'immagine)
- Anche se sappiamo che stiamo aspettando una JSON string, che presumibilmente arriva in tempi brevi, dobbiamo utilizzare il metodo **res.json()** per metterci in attesa che i dati vengano interamente trasferiti e quindi **deserializzati**.

Comunicazione con la server-side app

server-side API

Lo scopo della API è **esprimere le funzionalità** offerte dalla app lato server.

- Come le si richiede tramite HTTPRequest
- Che tipo di HTTPResponse ci si può aspettare in risposta, e quali altri effetti ha l'invocazione della funzionalità (in particolare gli effetti sul Data Model).

HTTPRequest

METHOD

PATHNAME/ROUTE

BODY

HTTPResponse

BODY

Effects on Data Model

Server-Side API - I

Una server-side app (ossia, ai nostri fini, un'applicazione che risponde a richieste HTTP, ma lo fa scambiando principalmente **dati** in formato JSON) **espone un accesso al Data Model**.

Il Data Model è più naturalmente gestito lato server, perché è il server ad avere la possibilità di rendere persistenti i dati, comunicando con un database.(*)

Applicazioni più complesse possono costruire sopra al modello dati una logica di business più articolata, che elabora i dati del DB e costruisce su di essi dei servizi «live», che forniscono funzionalità complesse, calcolate combinando i dati grezzi presenti nel DB con informazioni circostanziali o provenienti da terze parti.

Quindi possiamo vedere la cose in questo modo: l'app lato client **usa le funzionalità** dell'app lato server. Abbiamo visto che perché un'applicazione possa usare le funzionalità di un altro software (un'altra app, una libreria, un servizio online...) quest'ultimo deve dichiarare la propria API = Application Programming Interface.

In astratto una API è fatta di **funzionalità** che possono essere invocate; per ciascuna l'API dovrà dichiarare

- Come si invoca proprio quella funzionalità e non un'altra
- Quali input devono e/o possono essere forniti
- Quali sono gli effetti attesi e in particolare quali output vengono restituiti al chiamante

(*) non è impossibile per un'applicazione client-side rendere i dati persistenti ma si tratterà comunque di una persistenza limitata ai dati del singolo utente sul un singolo dispositivo; questo rende molto difficile la condivisione delle informazioni fra utenti diversi e fra dispositivi diversi, dunque si tratta di una persistenza un po' limitata.

Server-Side API - II

Dunque, come si descrive un'API «costruita» sopra il protocollo HTTP?

Vediamo un elemento per volta.

Qualificare le funzionalità

Per specificare alla server-side app quale funzionalità vogliamo richiedere, dobbiamo usare ciò che la HTTPRequest ci mette a disposizione; in particolare si useranno:

- Il **pathname** della URL: ogni funzionalità è resa disponibile su un diverso pathname. Un pathname si compone di più segmenti separati da slash (/); una particolare sequenza di segmenti va a identificare una ben precisa funzionalità, ad esempio **/bookshelf/books** o **/bookshelf/collections** o ancora **/requests/expired** o anche **/login**. In questo contesto si parla anche di **route** proprio perché la sequenza di segmenti **instrada** verso una ben precisa funzionalità (**attenzione**: il concetto di **route** è applicabile anche lato client, se ne avremo la possibilità ne parleremo in seguito).
- Il **method**: è possibile avere più funzionalità disponibili sullo stesso pathname a patto che usino un **method** diverso. In questo corso vediamo come method soltanto GET e POST. In linea di massima si usa GET per le funzionalità che (per lo più) **forniscono** dati o risorse dal server verso il client, e POST per le funzionalità che (per lo più) permettono al client di inserire nuovi dati nel Data Model. Ad esempio, **GET /bookshelf/collections** potrà ottenere l'elenco delle raccolte di libri, mentre **POST /bookshelf/collections** potrà aggiungere una nuova raccolta. Si tratta di una semplificazione; non tutte le funzionalità cadono in una di queste categorie. In quel caso si sceglierà quale **method** usare in base all'input che la funzionalità richiede, e per il quale rimandiamo alla discussione seguente.

Server-Side API - III

Esprimere gli input

Una funzionalità server-side che richieda degli input dovrà specificare come essi devono essere espressi all'interno della richiesta HTTP. Potrà farlo utilizzando tre diversi aspetti della HTTPRequest:

- Il **body** della Request, che però è disponibile solo per le richieste di tipo POST(*); tipicamente si usa per funzionalità che necessitano di un input «corposo» e strutturato, come ad esempio un array di valori o un oggetto. D'altra parte sono tipicamente le richieste di tipo POST a necessitare di questo tipo di input.
- La **query string**. Piccolo ripasso dalla prima parte del corso:

Query string

Si tratta di un testo opzionale che aggiunge ulteriori informazioni alla richiesta, sotto forma di **coppie chiave (c)/valore (v)**. La query string inizia sempre con il carattere ? seguito dalle coppie **c=v** separate, se ce ne fosse più di una, da un &. Quindi ad esempio una query string può avere la forma:

`?min=5&max=100&sort=alphabetical`

Ciascuna coppia chiave/valore costituisce un input.

Questo modo di richiedere degli input può essere utilizzato sia in caso di GET che di POST. Anche se abbiamo visto che teoricamente interi oggetti o array possono essere codificati come stringhe tramite JSON, anzi, qualunque file può esser codificato come una stringa e dunque passati come valori in una query string (e in effetti non è infrequente vedere query string lunghissime!), è più «elegante» usare la query string quando gli input sono valori semplici o comunque poco strutturati.

(*) questo perché ci stiamo restringendo alle sole GET e POST, naturalmente ci sono altri **method** che prevedono un **body** nella Request.

Server-Side API - III

Esprimere gli input (continua)

- La **route**, o per meglio dire, alcuni **segmenti** della route, che, quando così utilizzata, viene detta **route parametrica**. Quello che succede è che alcuni segmenti del pathname sono variabili e corrispondono a un determinato input. Ad esempio, la server-side API potrebbe dichiarare che per ottenere i dati di un libro si può usare la route **GET /bookshelf/books/:id** dove il segmento **:id** è di volta in volta sostituito dall'id del libro di cui si vogliono i dati. Dunque l'app client-side potrà effettuare la sua HTTPRequest su **/bookshelf/books/12** se vuole i dati del libro con **id** pari a 12, oppure su **/bookshelf/books/5** se vuole i dati del libro con **id** pari a 5. Questa modalità di specifica degli input viene tipicamente utilizzata per input che identificano una specifica risorsa (id, nomi, ecc). Ricordiamo infatti che la URL nel suo complesso dovrebbe essere un «Resource Locator», come dice il suo stesso acronimo. Inoltre è una modalità che funziona se gli input in questione sono numeri o brevi stringhe.

Dichiarare gli effetti attesi e le informazioni inviate in risposta

Questa componente dell'espressione di una funzionalità offerta tramite API è per certi versi la più semplice: la server-side API dovrà dichiarare (tramite opportuna documentazione), per ciascuna funzionalità offerta, in cosa consiste effettivamente quella funzionalità, in particolare:

- Le modifiche al Data Model conseguenti all'esecuzione della funzionalità e in dipendenza degli input ricevuti.
- Come sono fatti i risultati dell'elaborazione che vengono eventualmente reinviati al chiamante tramite il **body** della HTTPResponse.